

Experiment 2

Aim - To understand Version Control System / Source Code Management, install git and create a GitHub account

THEORY

What Is Version Control?

A system called version control, sometimes referred to as source control or revision control, keeps track of changes made to a file or group of files over time so that you may retrieve particular versions at a later time. Although it can be applied to any circumstance where several versions of something are made and may need to be monitored and recalled, it is most frequently employed in software development.

What is GIT?

- a. Git is a free and open-source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.
- b. Git relies on the basis of distributed development of software where more than one developer may have access to the source code of a specific application and can modify changes to it that may be seen by other developers..
- c. Initially designed and developed by Linus Torvalds for Linux kernel development in 2005.
- d. Every git working directory is a full-fledged repository with complete history and full version tracking capabilities, independent of network access or a central server.
- e. Git allows a team of people to work together, all using the same files. It helps the team cope with the confusion that tends to happen when multiple people are editing the same files.

What Is GitHub?

GitHub is an web based platform which hosts software development projects and uses Git for version management. Git is a distributed version control system that helps developers to work together on same software projects and keep track of changes made to their code by on another. GitHub offers a user-friendly interface, which is very collaborative tools, and more project management tools, GitHub will enhance the potential of the Git.

GitHub allows developers to create and manage the code in the repository in the remote location where others can access the code or Github is an collection repositories which contains the files of the project.

What is Use Of Version Control Software?

- a. Version control software allows the user to have “versions” of a project, which how the changes that were made to the code over time, and allows the user to backtrack if necessary and undo those changes.
- b. This ability alone– of being able to compare two versions or reverse changes, takes it fairly invaluable when working on larger projects.
- c. In a version control system, the changes would be saved just in time– a patch file that could be applied to one version, in order to make it the same as the next version.
- d. All versions are stored on a central server, and individual developers checkout and upload changes back to this server.

What Are the Uses Cases Of GitHub?

Following are some of the use cases of GitHub

- a. **Version Control:** GitHub is also called an version control system because of it uses such has if the certain developers are working on the same project and if any developer make changes at it is effecting the entire code then they move back to previous version with immediate actions.
- b. **Collaboration and Code Review:** GitHub allows group of developers work on same project where there can review the each others code and can work on the same project which will improve the productivity and where they can develop the complex application in faster manner.
- c. **Issue Tracking:** Has an certain group of developers will work on the same project so when the issue arises in the GitHub then you can assign the issue to the other developer to whom you want.
- d. **Open Source Development:** The most widely used platform for open source development is GitHub.

What Are Characteristics of Git?

A. Strong support for non-linear development

- a. Git supports rapid branching and merging and includes specific tools for visualizing and navigating a non-linear development history.
- b. A major assumption in Git is that a change will be merged more often than it is written.
- c. Branches in Git are very Lightweight.

B. Distributed development

- a. Git provides each developer a local copy of the entire development history, and changes are copied from one such repository to another.
- b. The changes can be merged in the same way as a locally developed branch very efficiently and effectively.

C. Compatibility with existing systems/protocol

- a. Git has a CVS server emulation, which enables the use of existing CVS clients and IDE plugins to access Git repositories.

D. Efficient handling of large projects

- a. Git is very fast and scalable compared to other version control systems.
- b. The fetching power from a local repository is much faster than is possible

with a remote server.

E. Data Assurance

- a. The Git history is stored in such a way that the ID of a particular version depends upon the complete development history leading up to that commit.
- b. Once published, it is not possible to change the old versions without them Being noticed.

F. Automatic Garbage Collection

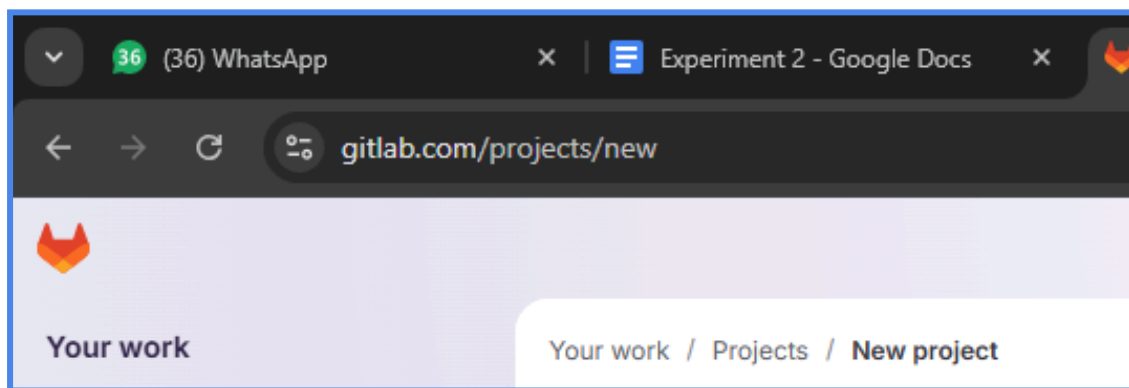
- a. Git automatically performs garbage collection when enough loose objects have been created in the repository.
- b. Garbage collection can be called explicitly using `git gc--prune`.
- c. Periodic explicit object packing
- d. Git stores each newly created object as a separate file. It uses packs that store a large number of objects in a single file (or network byte stream) called packfile, delta-compressed among themselves.
- e. A corresponding index file is created for each pack file, specifying the offset of each object in the packfile.
- f. The process of packing can be very expensive computationally.
- g. Git allows the expensive pack operation to be deferred until later when time does not matter.
- h. Git does periodic repacking automatically but manual repacking can be done with the `git gc` command.

How does GIT work?

- a. A Git repository is a key-value object store where all objects are indexed by their SHA-1 hash value.
- b. All commits, files, tags, and filesystem tree nodes are different types of objects living in this repository.
- c. A Git repository is a large hash table with no provision made for hash collisions.
- d. Git specifically works by taking “snapshots” of files

Steps to creating a Github account:

1. Go to <https://github.com/join> in a web browser.



2. Enter your personal details
3. Click Verify to start the verification puzzle
4. Click the green Create account button
5. Verify your email by entering the code
6. Select your preferences and click Continue
7. Note the types of plans offered by GitHub
8. Select the free plan

CONCLUSION-Thus we learnt about how to create a Github Account.

```

Last login: Wed Apr 15 22:34:22 on console
/dev/fd/12:25: command not found: compdef
[(base) mohitrajpal@Mohits-MacBook-Air ~ % git version
git version 2.51.0
[(base) mohitrajpal@Mohits-MacBook-Air ~ % git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

grow, mark and tweak your common history
  backfill   Download missing objects in a partial clone
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
[(base) mohitrajpal@Mohits-MacBook-Air ~ % mkdir t21/93
mkdir: t21: No such file or directory
[(base) mohitrajpal@Mohits-MacBook-Air ~ % mkdir t21-93
[(base) mohitrajpal@Mohits-MacBook-Air ~ % cd desktop
[(base) mohitrajpal@Mohits-MacBook-Air desktop % mkdir t21-93
[(base) mohitrajpal@Mohits-MacBook-Air desktop % cd t21-93
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global --list
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global user.email "mohitrajpal283@gmail.com"
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global --list
user.name=Mohit
user.email=mohitrajpal283@gmail.com

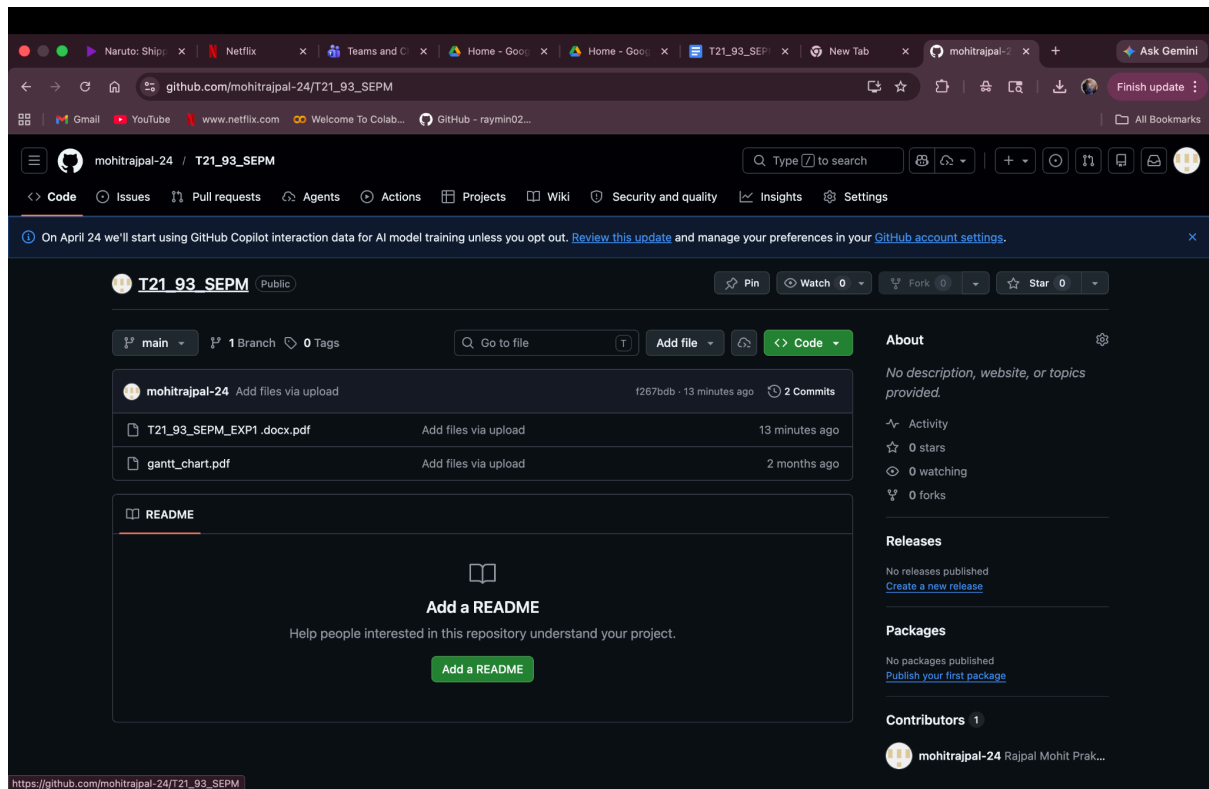
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/mohitrajpal/Desktop/t21-93/.git/
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 %

```

Name: Mohit Rajpal
93

Batch: T21

Roll No.




```
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git log --oneline
fatal: your current branch 'master' does not have any commits yet
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git commit -m "My second Commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git checkout - teststatus
error: pathspec '-' did not match any file(s) known to git
error: pathspec 'teststatus' did not match any file(s) known to git
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git checkout -- index.html
error: pathspec 'index.html' did not match any file(s) known to git
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git log
git log --oneline
git log --oneline index.html
fatal: your current branch 'master' does not have any commits yet
fatal: your current branch 'master' does not have any commits yet
fatal: ambiguous argument 'index.html': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git remote add origin https://github.com/mohitrajpal-24/T21_93_SEPM
[(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git push origin master
error: src refspec master does not match any
error: failed to push some refs to 'https://github.com/mohitrajpal-24/T21_93_SEPM'
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git log --oneline
git log --oneline index.html
fatal: your current branch 'master' does not have any commits yet
fatal: ambiguous argument 'index.html': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git push origin master
git pull origin master
git pull
git branch --set-upstream-to=origin/<branch> master
git log --oneline origin/master
git log --oneline master
error: src refspec master does not match any
error: failed to push some refs to 'https://github.com/mohitrajpal-24/T21_93_SEPM'
fatal: couldn't find remote ref master
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

```
Unpacking objects: 100% (6/6), 157.43 KiB | 1.05 MiB/s, done.
From https://github.com/mohitrajpal-24/T21_93_SEPM
* [new branch]      main      -> origin/main
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.
```

```
git pull <remote> <branch>
```

If you wish to set tracking information for this branch you can do so with:

```
git branch --set-upstream-to=origin/<branch> master
```

```
zsh: no such file or directory: branch
fatal: ambiguous argument 'origin/master': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
fatal: ambiguous argument 'master': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
(base) mohitrajpal@Mohits-MacBook-Air t21-93 %
```